# Sequential Search And Binary Search

***date:2025-03-02***

## chapter 1: Intorduction

In the field of computer science, search algorithms play a crucial role in efficiently retrieving information from data structures. Among the various search techniques, sequential search and binary search are two fundamental methods used to locate specific elements within a list of ordered integers. This project aims to explore and compare the performance of these two algorithms, particularly in their worst-case scenarios.

Sequential search, also known as linear search, is a straightforward approach that scans through the list from the beginning to the end, checking each element sequentially until the desired value is found or the end of the list is reached. While simple to implement, this method can be inefficient for large datasets, especially in the worst-case scenario where the target element is either not present in the list or located at the very end.

On the other hand, binary search is a more sophisticated algorithm that leverages the sorted nature of the list to significantly reduce the number of comparisons needed. By repeatedly dividing the search interval in half, binary search can quickly narrow down the location of the target element. However, its efficiency is contingent upon the list being sorted, and its worst-case scenario occurs when the target element is not present in the list, requiring the algorithm to traverse the entire search space.

The primary objective of this project is to analyze and compare the worst-case performance of sequential search and binary search when applied to a list of ordered integers. By understanding the strengths and limitations of each algorithm, we can gain insights into their practical applications and determine which method is more suitable for specific use cases. This exploration will not only enhance our understanding of search algorithms but also provide valuable knowledge for optimizing data retrieval processes in real-world applications.

# chapter 2: Algorithm Specification

**Input:**generate a random array and sort it to replace the input

**output:**the running time of each algorithm

**algorithm1**: sequential search

**Main idea:**Scan the list from the beginning to the end, checking each element one by one until the target is found or the end of the list is reached.

**pseudo code:**

```
int SequentialSearch(list, target):
for each element IN list:
if element == target:
return index of element
// if found,return index
return -1 // if not found,return -1
```

**algorithm2:**binary search

**Main idea:**Repeatedly divide the sorted list into two halves and eliminate the half where the target cannot reside, based on a comparison with the middle element, until the target is found or the search space is exhausted.

**pseudo code:**

```
int BinarySearch(list, target):
low = 0
high = length(list) - 1
while low <= high:
mid = (low + high) / 2 // calculate the mid index
if list[mid] == target:
return mid // if found,return index
else if list[mid] < target:
low = mid + 1 // the target is in the right part
else:
high = mid - 1 // the target is in the left part
return -1 // if not found, return -1
```

# chapter 3:Testing Results

The following table summarizes the test cases used to verify the performance of the four search algorithms: Binary Search (Iterative), Binary Search (Recursive), Sequential Search (Iterative), and Sequential Search (Recursive). Each test case includes the iterations,ticks,total time and durations.

Table1:Test cases for the four search algorithms

| | N | 100 | 500 | 1000 | 2000 | 4000 | 6000 | 8000 | 10000 |
|---|---|---|---|---|---|---|---|---|---|
| Binary Search (iterative version) | Iterations(K) | 1,000 | 1,000 | 1,000 | 1,000 | 1,000 | 1,000 | 1,000 | 1,000 |
| | Ticks | 116 | 175 | 194 | 160 | 117 | 123 | 114 | 129 |
| | Total Time(sec) | 0.00116 | 0.00175 | 0.00194 | 0.0016 | 0.00117 | 0.00123 | 0.00114 | 0.00129 |
| | Duration(ms) | 0.00116 | 0.00175 | 0.00194 | 0.0016 | 0.00117 | 0.00123 | 0.00114 | 0.00129 |
| Binary Search (recursive version) | Iterations(K) | 1,000 | 1,000 | 1,000 | 1,000 | 1,000 | 1,000 | 1,000 | 1,000 |
| | Ticks | 125 | 201 | 227 | 189 | 139 | 146 | 134 | 158 |
| | Total Time (sec) | 0.00125 | 0.00201 | 0.00227 | 0.00189 | 0.00139 | 0.00146 | 0.00134 | 0.00158 |
| | Duration(ms) | 0.00125 | 0.00201 | 0.00227 | 0.00189 | 0.00139 | 0.00146 | 0.00134 | 0.00158 |
| Sequential Search (iterative version) | Iterations(K) | 1,000 | 1,000 | 1,000 | 1,000 | 1,000 | 1,000 | 1,000 | 1,000 |
| | Ticks | 274 | 1580 | 2429 | 4949 | 6159 | 12107 | 16615 | 20663 |
| | Total Time(sec) | 0.00274 | 0.0158 | 0.02429 | 0.04949 | 0.06159 | 0.12107 | 0.16615 | 0.20663 |
| | Duration(ms) | 0.00274 | 0.0158 | 0.02429 | 0.04949 | 0.06159 | 0.12107 | 0.16615 | 0.20663 |
| Sequential Search (recursive version) | Iterations(K) | 1,000 | 1,000 | 1,000 | 1,000 | 1,000 | 1,000 | 1,000 | 1,000 |
| | Ticks | 827 | 4753 | 12452 | 21845 | 36388 | 47101 | 58448 | 70458 |
| | Total Time(sec) | 0.00827 | 0.04753 | 0.12452 | 0.21845 | 0.36388 | 0.47101 | 0.58448 | 0.70458 |
| | Duration(ms) | 0.00827 | 0.04753 | 0.12452 | 0.21845 | 0.36388 | 0.47101 | 0.58448 | 0.70458 |

# chapter 4:Analysis and Comments

## 1. **Sequential Search (Iterative)**:

### **Time Complexity**:

a)In the worst case, the target element is either not present in the list or located at the last position. The algorithm must scan through all N elements.

b)Therefore, the worst-case time complexity is O(N).

c)The average-case time complexity is also O(N), as on average, the algorithm may need to check half of the elements.

### **Space Complexity**:

The algorithm uses a constant amount of extra space (for variables like index and target).Therefore, the space complexity is O(1).

## 2. Sequential Search (Recursive):

### Time Complexity:

Similar to the iterative version, the worst-case and average-case time complexity is O(N).

### Space Complexity:

The recursive version uses additional space on the call stack. In the worst case, the maximum depth of recursion is O(N).

## 3. Binary Search (Iterative):

### Time Complexity:

Binary search repeatedly divides the search interval in half. At each step, the algorithm reduces the problem size by a factor of 2.Therefore, the worst-case and average-case time complexity is O(logN).

### Space Complexity:

The iterative version uses a constant amount of extra space (for variables like low, high, and mid).Therefore, the space complexity is O(1).

## 4. Binary Search (Recursive):

### Time Complexity:

Similar to the iterative version, the worst-case and average-case time complexity is O(logN).However, the recursive version incurs additional overhead due to function calls.

## Space Complexity:

The recursive version uses additional space on the call stack. In the worst case, the maximum depth of recursion isO(logN).Therefore, the space complexity is O(logN).

Table2 is a summary of the complexities of the four algorithms.

Table2

| Algorithm | Time Complexity (Worst Case) | Space Complexity |
|---|---|---|
| Sequential Search (Iterative) | $O(N)$ | $O(1)$ |
| Sequential Search (Recursive) | $O(N)$ | $O(N)$ |
| Binary Search (Iterative) | $O(\log N)$ | $O(1)$ |
| Binary Search (Recursive) | $O(\log N)$ | $O(\log N)$ |

## summary:

Sequential Search is simple but inefficient for large datasets due to its linear time complexity. It is best suited for small lists.Binary Search is highly efficient for sorted lists, with logarithmic time complexity.

## comments:

a)If the input or the array is unsorted,we should sort it first or used sequential searth algorithm.

b)Besides,The iterative versions of both algorithms have better space complexity because they do not use the recursion call stack. In the recursive versions:For sequential search, the recursion depth can be up to n, leading to O(n) space complexity.For binary search, the recursion depth is logN, leading to O(logN) space complexity.In contrast, the iterative versions use only constant extra space O(1), making them more space–efficient.

# Appendix: Source Code (in C)

```c
#include<stdio.h>
#include<stdlib.h>
#include<time.h>

#define N 10000
#define MAX 20000
#define numloops 1000
```

```c
void arr_init(int num[]);//array
initialization
void arr_sort(int num[],int l,int r);//quick
sort for array
```

```
int LinearSearchIterative(int num[],int
target);//Iterative Linear Search
int LinearSearchRecursive(int num[],int
target,int index);//Recursive Linear Search
int BinarySearchIterative(int num[],int
target);//Iterative Binary Search
int BinarySearchRecursive(int num[],int
target,int low,int high);//Recursive Binary
Search
```

```
clock_t start,stop;
double duration;
```

```
int main(void)
{
//set the seed of the random numbers
srand((unsigned int)time(NULL));
int num[N];
arr_init(num); //generate a random sequence
arr_sort(num, 0 , N-1); //sort the array in
ascending order
// Test LinearSearchIterative
start = clock();
```

```
for(int i = 0;i < numloops;i++)
{
int target = rand() % MAX;
LinearSearchIterative(num, target);
}
stop = clock();
duration = ((double)(stop - start)) /
CLOCKS_PER_SEC; // calculate the total time of
the search
printf("Total ticks: %ld ticks;Total time: %f
seconds;Average time per search: %f
milliseconds.\n",
stop-start,duration,1000*duration/numloops)
;
// Test LinearSearchRecursive
start = clock();
for(int i = 0;i < numloops;i++)
{
int target = rand() % MAX;
LinearSearchRecursive(num, target,0);
}
stop = clock();
```

```
duration = ((double)(stop - start)) /
CLOCKS_PER_SEC; // calculate the total time of
the search
printf("Total ticks: %ld ticks;Total time: %f
seconds;Average time per search: %f
milliseconds.\n",
stop-start,duration,1000*duration/numloops)
;
// Test BinarySearchIterative
start = clock();
for(int i = 0;i < numloops;i++)
{
int target = rand() % MAX;
BinarySearchIterative(num, target);
}
stop = clock();
duration = ((double)(stop - start)) /
CLOCKS_PER_SEC; // calculate the total time of
the search
printf("Total ticks: %ld ticks;Total time: %f
seconds;Average time per search: %f
milliseconds.\n",
```

```
stop-start,duration,1000*duration/numloops)
;
// Test BinarySearchRecursive
start = clock();
for(int i = 0;i < numloops;i++)
{
int target = rand() % MAX;
BinarySearchRecursive(num,target,0,N - 1);
}
stop = clock();
duration = ((double)(stop - start)) /
CLOCKS_PER_SEC; // calculate the total time of
the search
printf("Total ticks: %ld ticks;Total time: %f
seconds;Average time per search: %f
milliseconds.\n",
stop-start,duration,1000*duration/numloops)
;
return 0;
}
```

```
void arr_init(int num[])
{
```

```
//implement random array generation
int is_used[MAX] = {0};//index array
int count = 0;
while (count < N)//generate N numbers randomly
{
int x = rand() % MAX ;
if(is_used[x] == 0) //determine whether x is
already in the array
{
is_used[x] = 1; //mark the element of the index
array
num[count] = x; //
count++;
}
else
continue;//if x is already in the
array,regenerate x
}
}
```

```
void arr_sort(int a[],int l,int r)
{
```

```
//quick sort
if(l>=r)return;
int x = a[l],i = l - 1,j = r + 1,t;
while(i<j)
{
do i++;while(a[i] < x);
do j--;while(a[j] > x);
if(i<=j){t = a[j];a[j] = a[i];a[i] = t;}
}
//recursive realization
arr_sort(a,l,j);
arr_sort(a,j+1,r);
}
```

```
int LinearSearchIterative(int num[],int
target)
{
//iterative linear search
for(int i = 0;i < N;i++)
{
if(num[i] == target)
return i;//return the index of the target
```

```
}
return -1;//if not found the target,return -1
}
```

```
int LinearSearchRecursive(int num[],int
target,int index)
{
//recursive linear search
if(index==N)return -1;//recursive exit
if(num[index]==target)return index;//return
the index of target
return
LinearSearchRecursive(num,target,index+1);/
/recursive
}
```

```
int BinarySearchIterative(int num[],int
target)
{
//iterative binary search
int low = 0;
int high = N - 1;
while(low <= high)
```

```
{
int mid = (low + high)/2;
if(num[mid] == target)
return mid;
else if(num[mid] > target)
high = mid - 1;//change to the left interval
else
low = mid + 1;//change to the right interval
}
return -1;//if not found,return -1
}
```

```
int BinarySearchRecursive(int num[],int
target,int low,int high)
{
//recursive binary search
if(low > high)return -1;//recursive exit
int mid = (low + high)/2;
if(num[mid] == target)return mid;
else if(num[mid] > target)return
BinarySearchRecursive(num,target,low,mid-1);
//recursive in the left interval
```

```
else return
BinarySearchRecursive(num,target,mid+1,high)
;//recursive in the right interval
}
```

# Declaration

*I hereby declare that all the work done in this project is of my independent effort.*